

Segmentation

A user views a program as a collection of segments such as main program, routines, variables, etc.

All of these segments are variable in size and their size may also vary during execution.

Each segment is identified by a name (or segment number) and the elements within a segment are identified by their offset from the starting of the segment. Figure 7.21 shows the user view of a program.

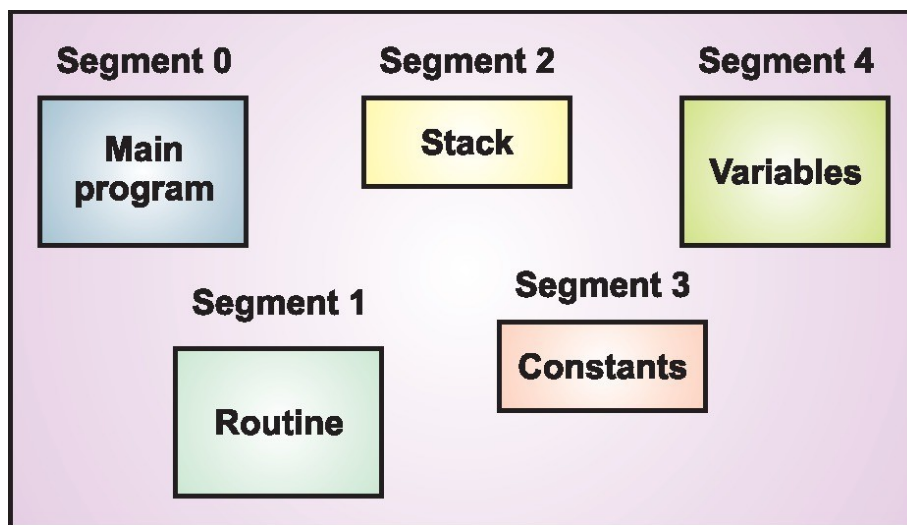


Fig. 7.21 User View of a Program

Segmentation is a memory management scheme that implements the user view of a program. I

n this scheme, the entire logical address space is considered as a collection of segments with each segment having a number and a length. The length of a segment may range from 0 to some maximum value as specified by the hardware and may also change during the execution.

The user specifies each logical address consisting of a segment number (s) and an offset (d).

This differentiates segmentation from paging in which the division of logical address into page number and page offset is performed by the hardware.

To keep track of each segment, a **segment table** is maintained by the operating system (see [Figure 7.22](#)). Each entry in the segment table consists of two fields: **segment base** and **segment limit**.

The **segment base** specifies the starting address of the segment in physical memory and the **segment limit** specifies the length of the segment. The segment number is used as an index to the segment table.

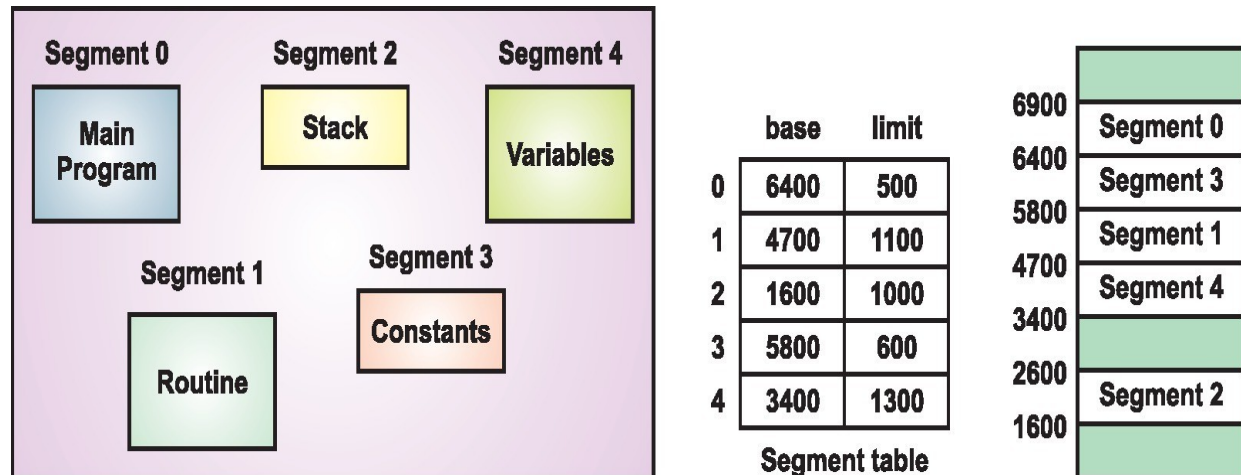


Fig. 7.22 A Segment Table

When CPU generates a logical address, that address is sent to MMU. The MMU uses the segment number of logical address as an index to the segment table. The offset is compared with the segment limit and if it is greater, invalid-address error is generated. Otherwise, the offset is added to the segment base to form the physical address that is sent to the memory. [Figure 7.23](#) shows the hardware to translate logical address into physical address in segmentation.

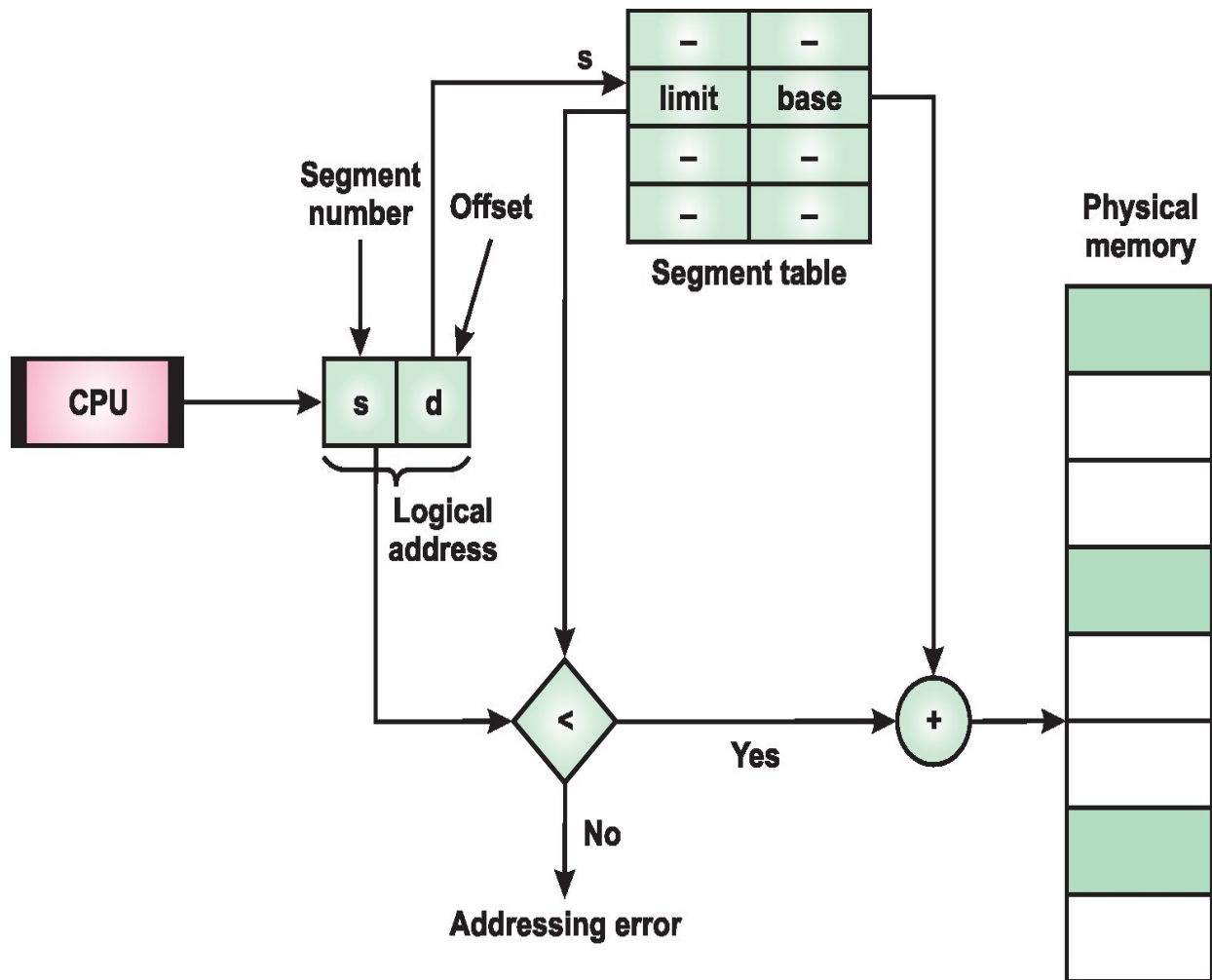


Fig. 7.23 Segmentation Hardware

Advantages

- Since a segment contains one type of object, **each segment can have different type of protection**. For example, a procedure can be specified as execute only whereas a char type array can be specified as read only.
- **It allows sharing of data or code between several processes**. For example, a common function or shared library can be shared between various processes. Instead of having them in address space of every process, they can be put in a segment and that segment can be shared.

Example 8 Using the following segment table, compute the physical address for the logical address consisting of segment and offset as given below.

(a) segment 2 and offset 247

(b) segment 4 and offset 439

	Base	Limit
0	5432	350
1	115	100
2	2200	780
3	4235	1100
4	1650	400

Segment table

Solution

(a) Here, offset = 247 and segment is 2.

It is clear from the segment table that the limit of segment 2 is 780 and that of segment base is 2200.

Since the offset is less than the segment limit, physical address is computed as:

Physical address = Offset + Segment base

$$= 247 + 2200 = 2447$$

(b) Here, Offset = 439 and segment is 4.

It is clear from the segment table that the limit of segment 4 is 400 and that of segment base is 1650.

Since the offset is greater than the segment limit, invalid-address error is generated.